CCAI - 311

19/11/2025

المملكة العربية السعودية

وزارة التعليم

جامعة جدّة

كلية علوم و هندسة الحاسب

Optimization Project Report

Students work :

1 – Abdullah Alharbi 2340083

2 – Mohammed Darandri 2340123

3 – Muhannad Alzahrani 2340114

4 – Rakan fawzi jalalah 2340267

Supervision of Dr : Mohammed Dessokey

1. Introduction

Optimization is an essential tool in modern decision-making, allowing organizations to maximize profit, minimize cost, or use resources efficiently.
In this project, we formulate a real linear programming (LP) problem, transform it into its dual form, and solve it using two optimization methods:

1. **Exact Method** (Simplex – HiGHS Solver)
2. **Approximate Method** (Heuristic Random Search)

The purpose is to evaluate how different optimization approaches perform and compare their effectiveness in solving real-world problems.

---

2. Problem Formulation (Part 1)

2.1 Real-World Scenario

A factory produces two products: Product A and Product B.
Both products require machine hours and labor hours.
The company wants to **maximize total profit** while staying within the available resources.

Profit per unit

- Product A → 60 SAR
- Product B → 50 SAR

Resource availability

- Machine hours: 100 hours
- Labor hours: 90 hours

Resource usage per unit

Product	Machine Hours	Labor Hours
A (x1)	4	1
B (x2)	2	3

2.2 Decision Variables

$$x_1 = \text{units of Product A}$$
$$x_2 = \text{units of Product B}$$

2.3 Objective Function (Profit Maximization)

$$\max\ Z = 60x_1 + 50x_2$$

2.4 Constraints

Machine Hours Constraint

$$4x_1 + 2x_2 \leq 100$$

Labor Hours Constraint

$$x_1 + 3x_2 \leq 90$$

Non-negativity

$$x_1, x_2 \geq 0$$

3. Dual Problem (Transformation)

We introduce dual variables:

- y_1= shadow price of machine hours
- y_2= shadow price of labor hours

3.1 Dual Objective Function

$$\min\ W = 100y_1 + 90y_2$$

3.2 Dual Constraints

$$\begin{aligned} &4y_1 + 1y_2 \geq 60 \\ &2y_1 + 3y_2 \geq 50 \\ &y_1, y_2 \geq 0 \end{aligned}$$

This completes the required **Primal → Dual transformation**.

---

4. Optimization Methods Implementation (Part 2)

Two methods were implemented: an exact algorithm and a heuristic one.

4.1 Exact Method – Simplex (HiGHS Solver)

Implemented using **SciPy's linprog**, which guarantees the optimal solution for linear programming.

Initial Solution

The solver starts with a basic feasible solution generated internally by HiGHS.

Optimal Solution

The exact solver produced:

- $x_1 = 12$
- $x_2 = 26$
- $Z = 2020$ SAR

Verification of Optimality

- All constraints satisfied
- Non-negativity satisfied
- Objective cannot be further improved
- Strong Duality holds → This confirms the optimality of the primal solution

---

4.2 Approximate Method – Random Search

A heuristic method that:

1. Generates random values for x_1, x_2
2. Checks feasibility
3. Computes the profit
4. Tracks the best solution found across iterations

Approximate Method Result

- $x_1 = 11$
- $x_2 = 26$
- $Z \approx 1960$ SAR

This solution is near-optimal but not guaranteed to be the global optimum.

5. Results Summary

Method	x1	x2	Profit Z	Optimal?
Exact (Simplex)	12	26	**2020 SAR**	Yes
Approximate	11	26	**≈ 1960 SAR**	Not guaranteed

---

6. Analysis & Comparison

Exact Method

- Always returns the mathematically optimal solution
- Deterministic: produces the same result every time
- Extremely fast for linear problems
- Suitable for industrial applications

Approximate Method

- Depends on randomness and number of iterations
- Produces solutions close to the optimum
- Useful when the problem is nonlinear or too complex for traditional solvers
- May vary between runs

Comparison Table

Criterion	Exact (Simplex)	Approximate (Random Search)
Accuracy	Optimal	Near-optimal
Stability	High	Medium
Speed	Very fast	Medium (depends on iterations)
Guarantees	Yes	No
Use case	Linear problems	Nonlinear/large problems

---

7. Conclusion

This project successfully formulated and solved a linear programming problem using both exact and approximate optimization methods.

Key findings:

- The **Exact Method** (Simplex) provided the true optimal profit of **2020 SAR**.
- The **Approximate Method** generated a profit close to the optimal, demonstrating its usefulness for heuristic exploration.
- The dual transformation confirmed the correctness of the primal solution through dual feasibility and strong duality.
-

---

8. Future Recommendations

- Apply more advanced meta-heuristics such as Genetic Algorithms or Simulated Annealing.
- Extend the problem to multiple time periods or additional product types.
- Integrate cost minimization scenarios for comparison.
-

9. Documented Code (Exact & Approximate Methods)

Exact Method Code

```
import numpy as np
from scipy.optimize import linprog

c = [-60, -50]
A = [
    [4, 2],
    [1, 3]
]
b = [100, 90]

x_bounds = (0, None)
bounds = [x_bounds, x_bounds]

res = linprog(c, A_ub=A, b_ub=b, bounds=bounds,
method="highs")

if res.success:
    x1, x2 = res.x
    Z = 60*x1 + 50*x2
    print("Optimal solution (Exact):")
    print(f"x1 = {x1:.2f}, x2 = {x2:.2f}")
    print(f"Max Profit Z = {Z:.2f} SAR")
else:
    print("No optimal solution found.")
```

```
res = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method="highs")

if res.success:
    x1, x2 = res.x
    Z = 60*x1 + 50*x2
    print("Optimal solution (Exact):")
    print(f"x1 = {x1:.2f}, x2 = {x2:.2f}")
    print(f"Max Profit Z = {Z:.2f} SAR")
else:
    print("No optimal solution found.")
```

```
Optimal solution (Exact):
x1 = 12.00, x2 = 26.00
Max Profit Z = 2020.00 SAR
```

Approximate Method Code

```
import random

def Z(x1, x2):
     return 60*x1 + 50*x2

 def valid(x1, x2):
     return (
         4*x1 + 2*x2 <= 100 and
         x1 + 3*x2 <= 90 and
        x1 >= 0 and x2 >= 0
    )

best_x1, best_x2 = 0, 0
best_Z = -1

for _ in range(5000):
    x1 = random.randint(0, 100)
    x2 = random.randint(0, 100)

    if valid(x1, x2):
        current_Z = Z(x1, x2)
        if current_Z > best_Z:
            best_Z = current_Z
            best_x1, best_x2 = x1, x2

print("Approximate Method Result:")
print(f"x1 = {best_x1}, x2 = {best_x2}")
print(f"Z ≈ {best_Z}")
```

```
print("Approximate Method Result:")
print(f"x1 = {best_x1}, x2 = {best_x2}")
print(f"Z ≈ {best_Z}")
```

```
Approximate Method Result:
x1 = 11, x2 = 26
Z ≈ 1960
```

Solver Solution

G	F	E	D	C	B
	X1	X2	Z		
	12	26	2020		
Solution					
Obj. coeff	60	50			
			LHS		RHS
const 1	4	2	100	<=	100
const 2	1	3	90	<=	90